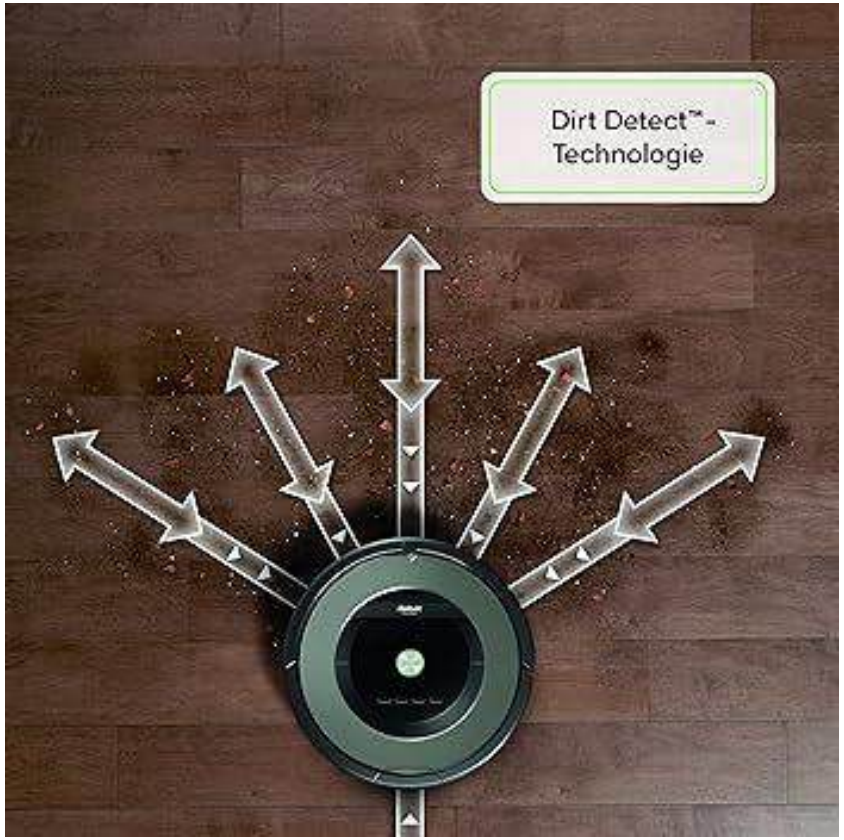
Laborprojekt Servicerobotik / Bildverarbeitung für Robotik 2025

Dirt Detection for Cleaning Robots

By Mahla and Danish

Introduction

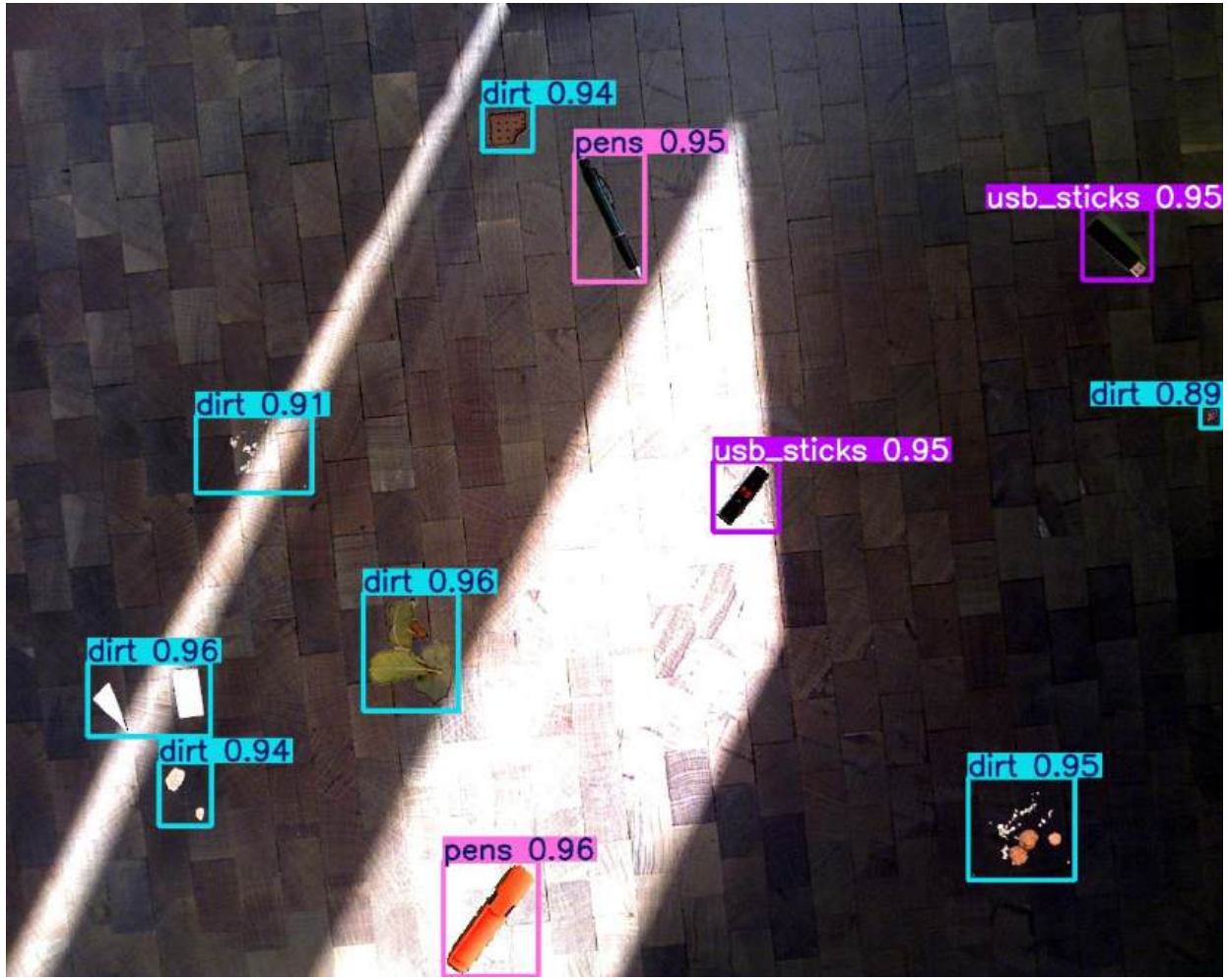
- **Current robots sense dirt mechanically:** Most consumer vacuums (e.g. Roomba's Dirt Detect™) use piezoelectric or optical sensors at the suction inlet to trigger extra passes when they "feel" more debris.
- **No camera-based dirt detection yet:** Off-the-shelf cleaners can't visually distinguish dust, stains or avoid everyday objects.



Source: IRobot from Amazon at 385 Euros

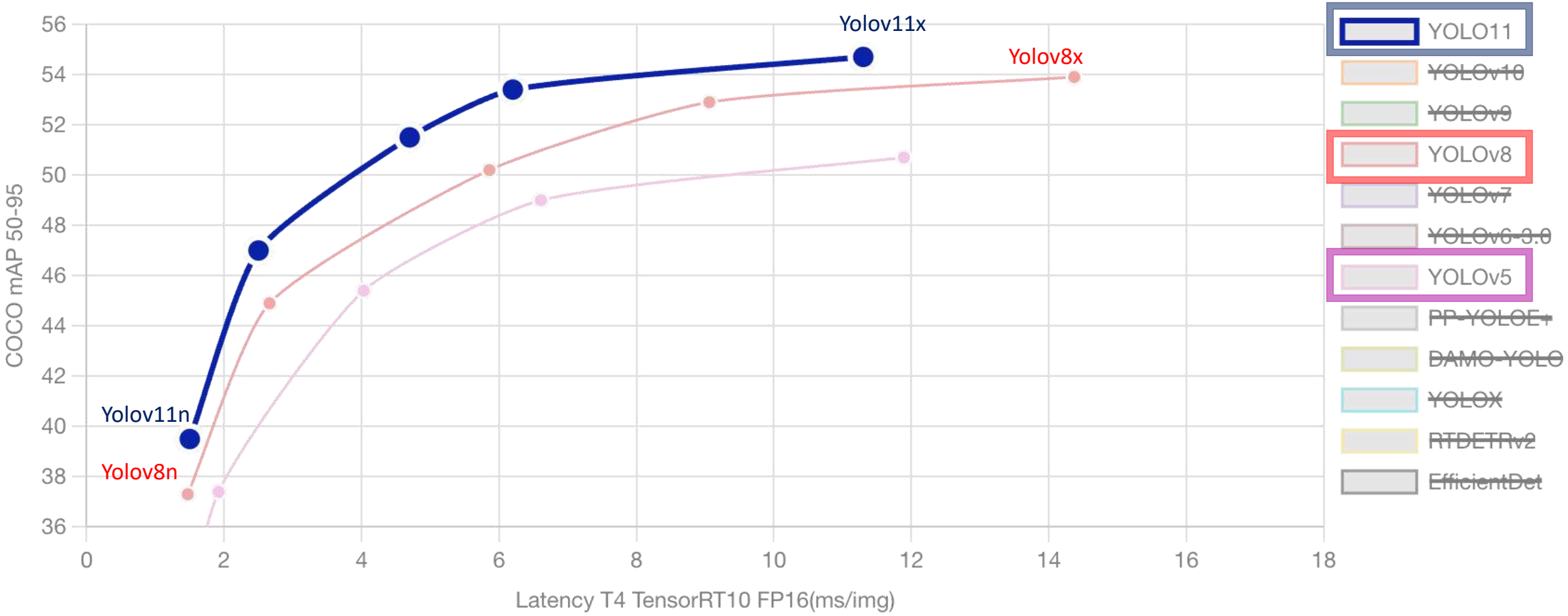
Motivation

- **Implement and evaluate** a **single-stage object detection model** for accurate dirt (dust, stains) detection
- **Distinguish** dirt from common office items (pens, books, keys etc)
- **Optimize** for real-time inference on standard hardware (CPU/GPU)



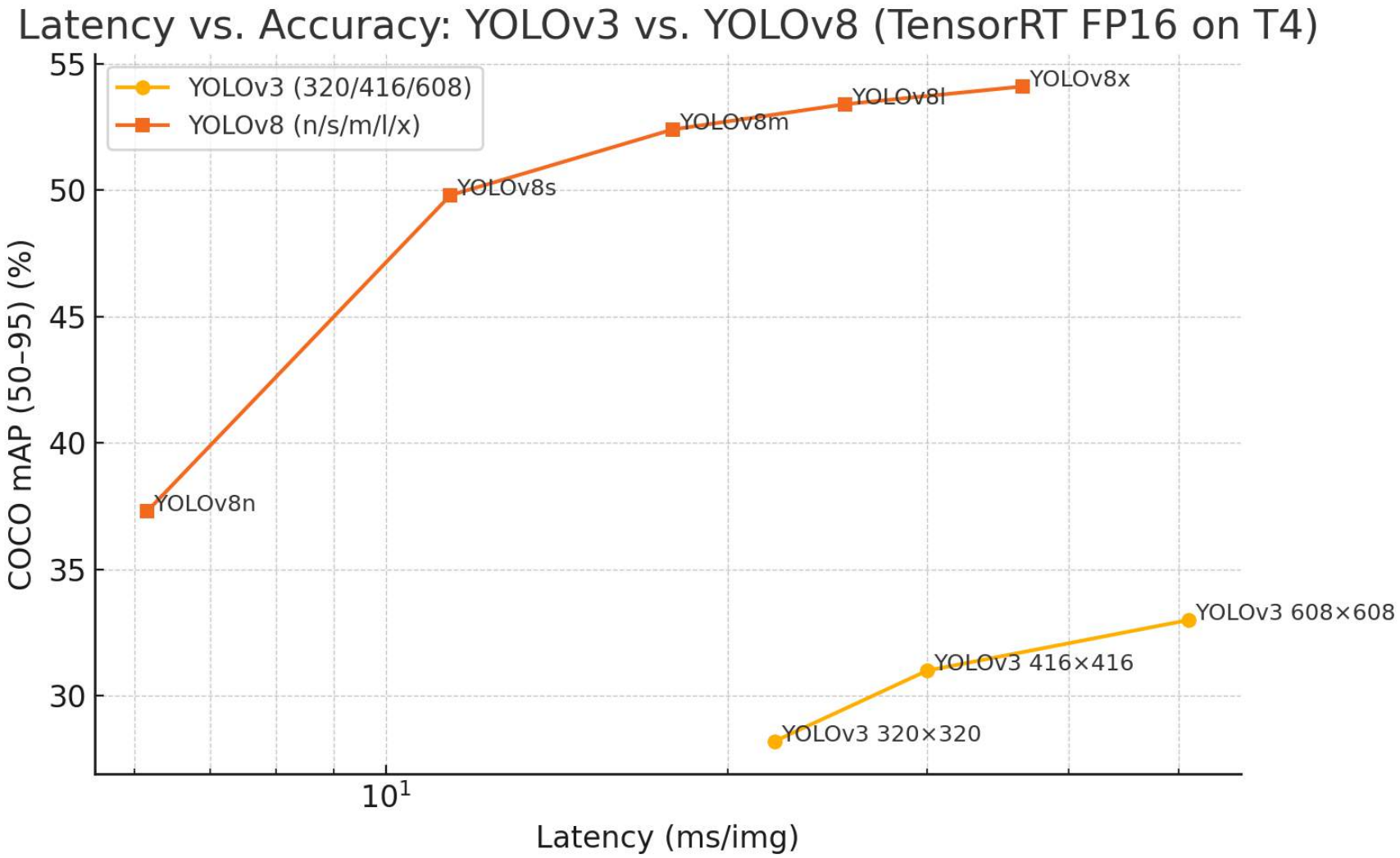
YOLOv8 trained model detection results

Comparison of different models



Source: https://docs.ultralytics.com/de/compare/

YOLOv3 vs YOLOv8



Source: Chat-GPT o4-mini-high

Technical implementation and Experiments

- **Model Selection & Deployment Plan**
- **First choice: YOLOv3**
  - Baseline implementation ran on macOS but hit dependency and label-format issues.
- **Second choice: YOLOv8**
  - Proven faster and more accurate in early google colab tests (even with minibatch 2).
- **Final setup**
  - **Hardware**: Remote box with 2 GPUs, multi-worker data loaders
  - **Data**: Large dataset + mosaic & standard augmentations
  - **Training**: Minibatch 16 (×4 via mosaic), mixed-precision (FP16), LR scheduling
  - **Validation**: Regular checkpoints to guard against overfitting

YOLOv3

- **Why YOLOv3?** Proposal in the Paper
- **YOLOv3 manual implementation (train.py)**
  Frequent layer-shape mismatches prevented use of the standard yolo train pipeline.
- **Convolution redesign**
  All 1×1 and 3×3 filters were realigned to exactly match Darknet-53's channel dimensions.
- **Label-file parsing failures**
  Despite generating YOLO-format .txt labels, the custom loader misread files multiple times, interrupting training.
- **Decision**
  Completely migrate to YOLOv8 for a robust, out-of-the-box training workflow and strict label validation

YOLOv8

- **Plug-and-play training**
  YOLOv8 provides a ready-made training pipeline—just install and run, no custom scripts required
- **Intuitive interface**
  A clear CLI and Python API make setup and experimentation straightforward for any user
- **Zero manual design**
  All network layers and connections are predefined, eliminating error-prone manual configuration
- **Built-in label validation**
  Native support for YOLO .txt formats catches formatting issues before training begins
- **Streamlined optimization**
  Integrated hyperparameter search and pruning tools speed up model tuning

YOLOv8 with CPU

- **Used CPU version:**

  (AMD-A12 core-Radeon R7)
- **Google Drive Integration**

  Uploading images to Google Drive
- **Preparing the YOLO-Dataset**

  Splitting images into training and validation sets

  Creating YOLO-format labels for each image
- **Generating `dataset.yaml`**

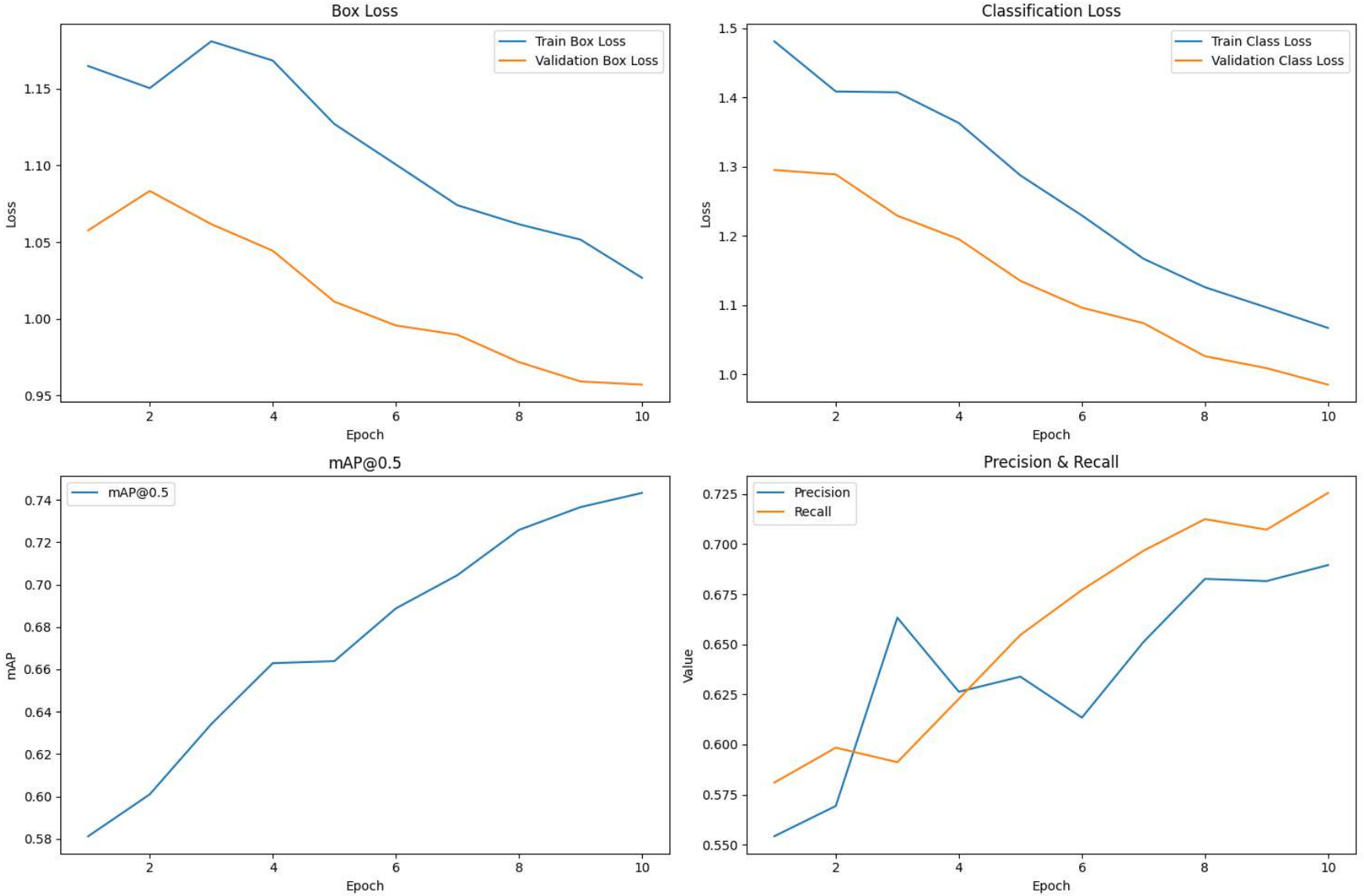
  Manually creating YAML file to define classes, label paths, and image paths
- **Model Training Setup**

  Training model without optimization or regularization

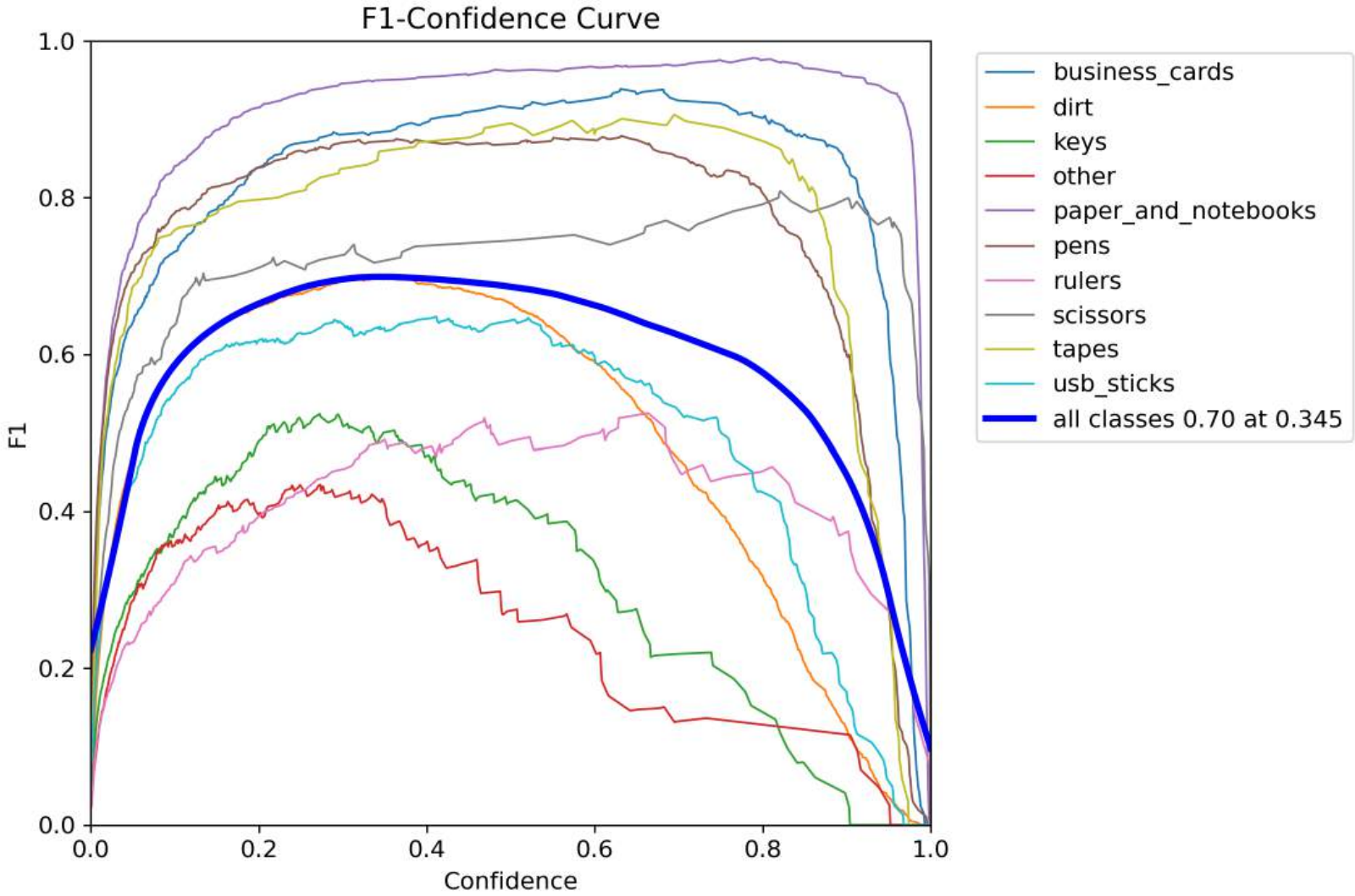
  Using small batch size and limited dataset
- **Training Results and Visualization**

  Generating plots to evaluate accuracy and training performance

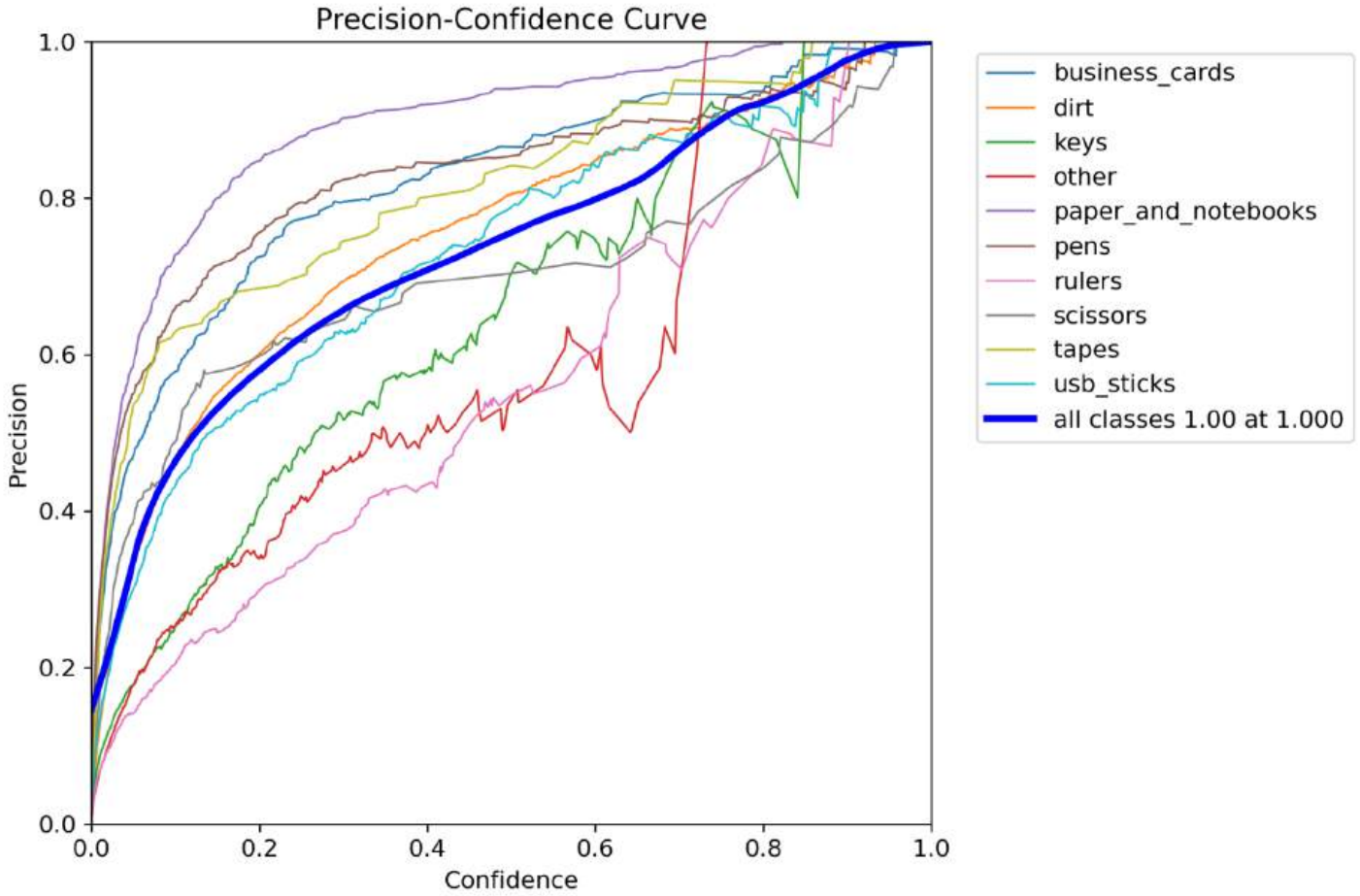
YOLOv8 - Loss & mAP



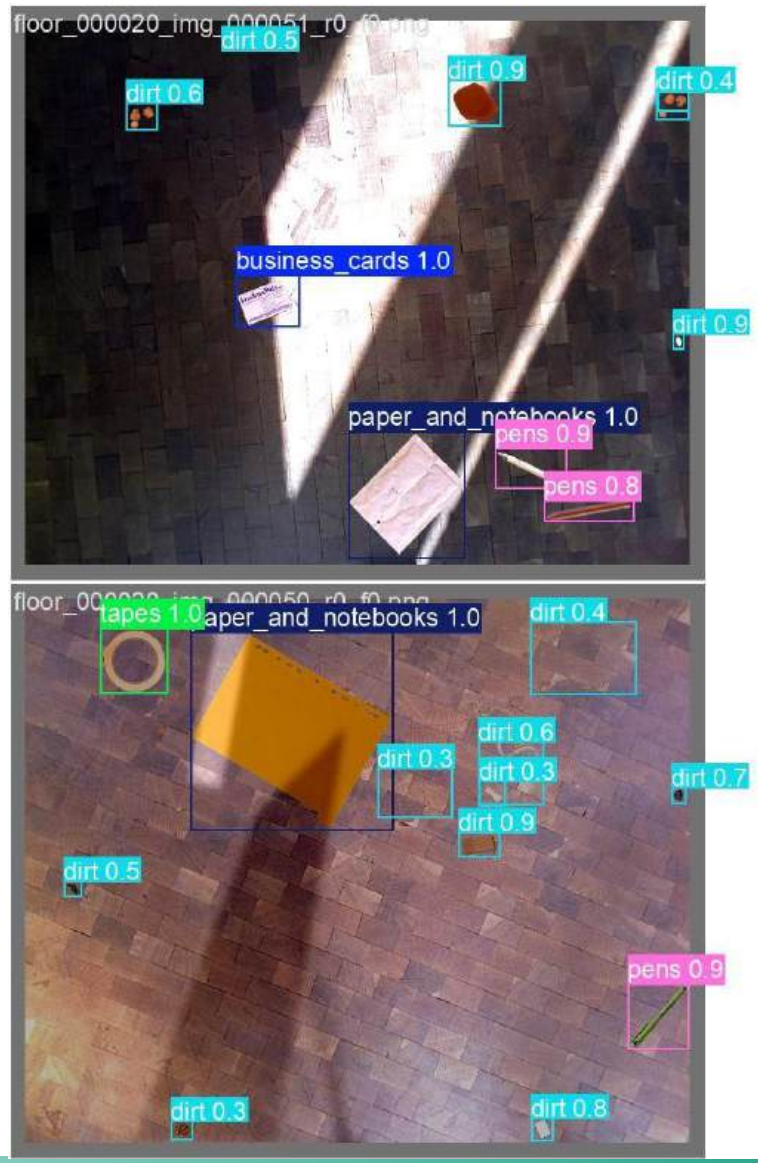
F1-Confidence Curve



Precision - Confidence Curve



Prediction/ Validation

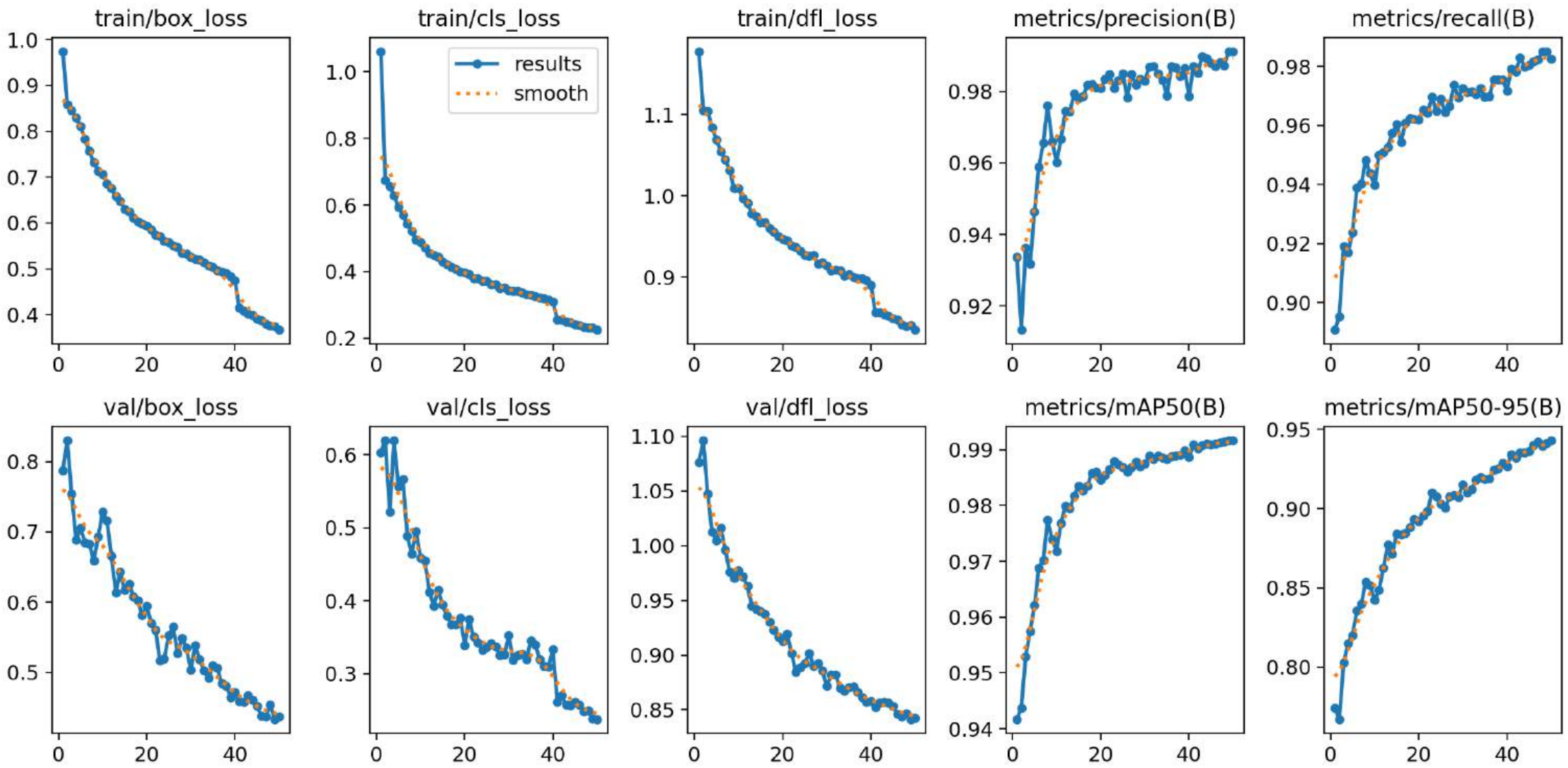


YOLOv8 with GPU

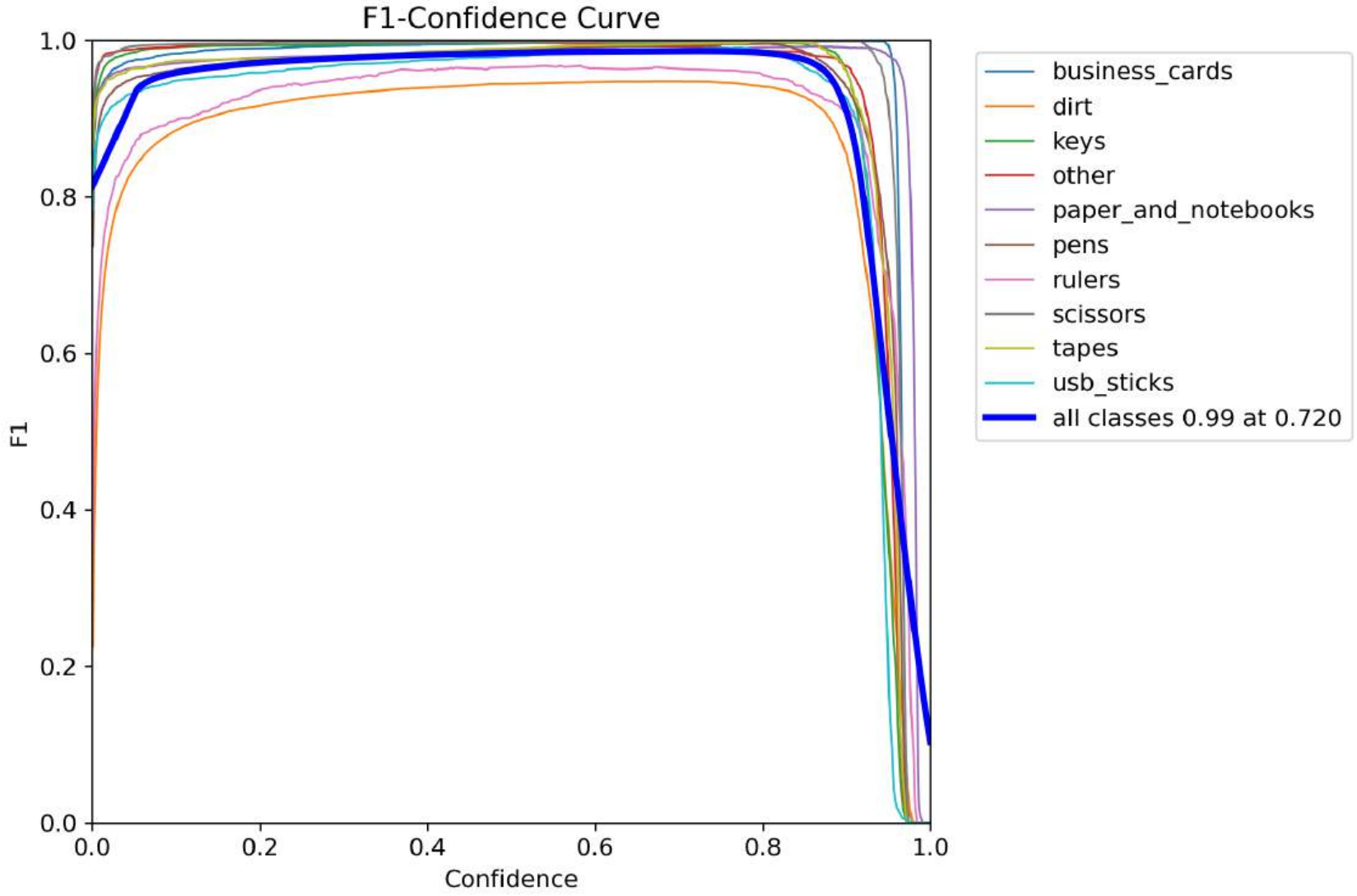
- **NVIDIA-SMI & Driver**
  Version: 550.144.03
  CUDA: 12.4
- **GPU 0: NVIDIA GeForce RTX 3090 Ti**
  Memory: 24.0 GB
- **GPU 1: NVIDIA GeForce RTX 3090**
  Memory: 24.0 GB

```
yolo train \
data=data/dirt.yaml \
model=yolov8x.pt \
device=0,1 \
workers=8 \
cache=disk \
amp=true \
imgsz=1024 \
batch=16 \
epochs=50 \
patience=10 \
lr0=0.001 \
lrf=0.1 \
warmup_epochs=5 \
mosaic=1.0 \
copy_paste=0.5 \
mixup=0.2 \
auto_augment=randaugment \
project=runs/train \
name=dirtnet_largest
```

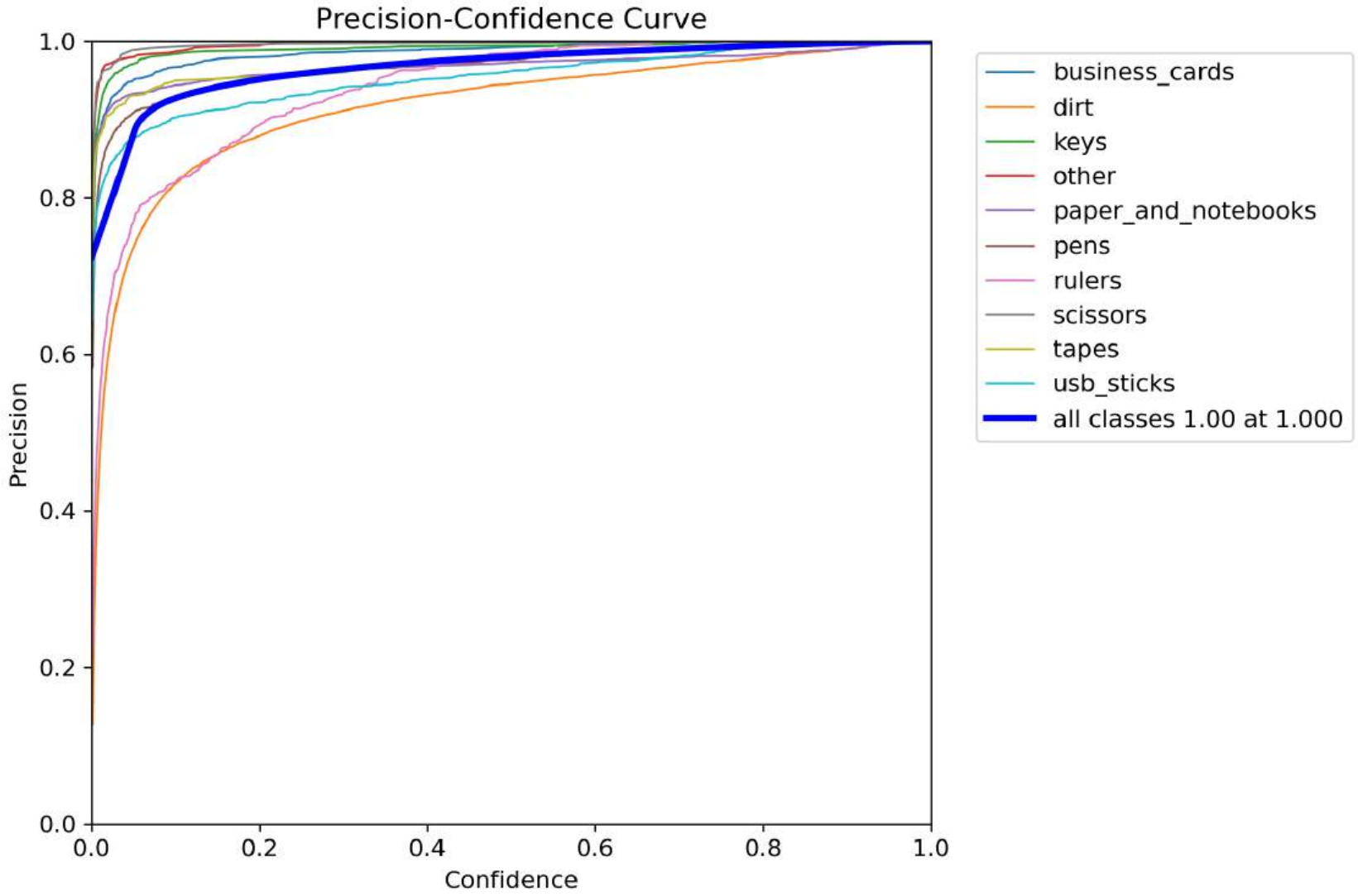
YOLOv8 - Loss & mAP



F1-Confidence Curve



Precision - Confidence Curve



Ground Truth

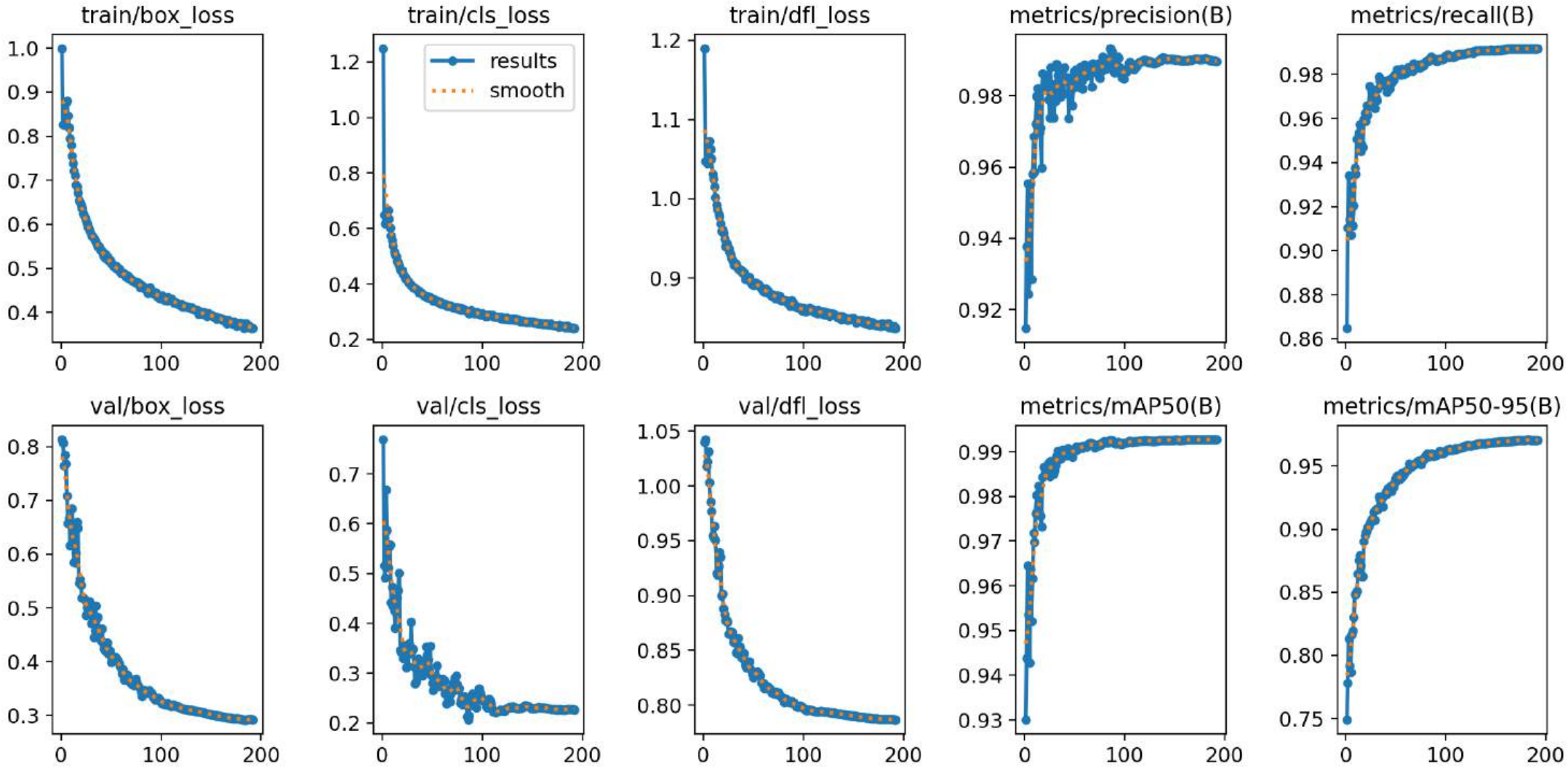
Prediction


dirt 0.7 dirt 0.9 dirt 0.3 dirt 1.0 dirt 0.5 dirt 0.9 paper_and_notebooks 0.4 business_cards 1.0 pens 0.3 rulers 1.0 business_cards 0.3
paper_and_notebooks 1.0 dirt 0.3 dirt 0.5 dirt 0.9 dirt 1.0 paper_and_notebooks 0.5 dirt 0.3 keys 0.9 pens 0.3
paper_and_notebooks 0.5 dirt 0.9 dirt 1.0 business_cards 1.0 dirt 0.5
paper_and_notebooks 0.8 dirt 0.4 scissors 1.0 dirt 0.6 dirt 0.3 business_cards 1.0 paper_and_notebooks 0.3
usb_sticks 0.9 dirt 1.0 business_cards 1.0 paper_and_notebooks 0.6 paper_and_notebooks 1.0 dirt 0.9 dirt 0.6 dirt 0.7 dirt 0.9 pens 1.0 scissors 1.0
usb_sticks 0.9 paper_and_notebooks 1.0 dirt 0.3 dirt 0.9 dirt 0.3 dirt 0.8 paper_and_notebooks 1.0 business_cards 1.0 dirt 0.4 dirt 0.9 dirt 0.4
paper_and_notebooks 0.5 usb_sticks 0.9
floor_000020_img_000061_r0 business_cards 1.0 dirt 0.3 dirt 0.9 dirt 1.0 usb_sticks 0.9 usb_sticks 0.5 paper_and_notebooks 0.4 keys 0.9 pens 1.0 keys 1.0 usb_sticks 0.6 dirt 0.4
paper_and_notebooks 0.3 floor_000020_img_000061_r1 dirt 0.9 dirt 0.4 paper_and_notebooks 0.5 paper_and_notebooks 1.0 notebooks 1.0 business_cards 0.3 dirt 0.5 paper_and_notebooks 0.9 paper_and_notebooks 0.4 business_cards 0.4 dirt 0.7
paper_and_notebooks 1.0 dirt 0.6 rulers 0.9 dirt 0.3 dirt 0.9 pens 1.0 usb_dirt 0.4 dirt 0.9 dirt 0.5 dirt 0.7

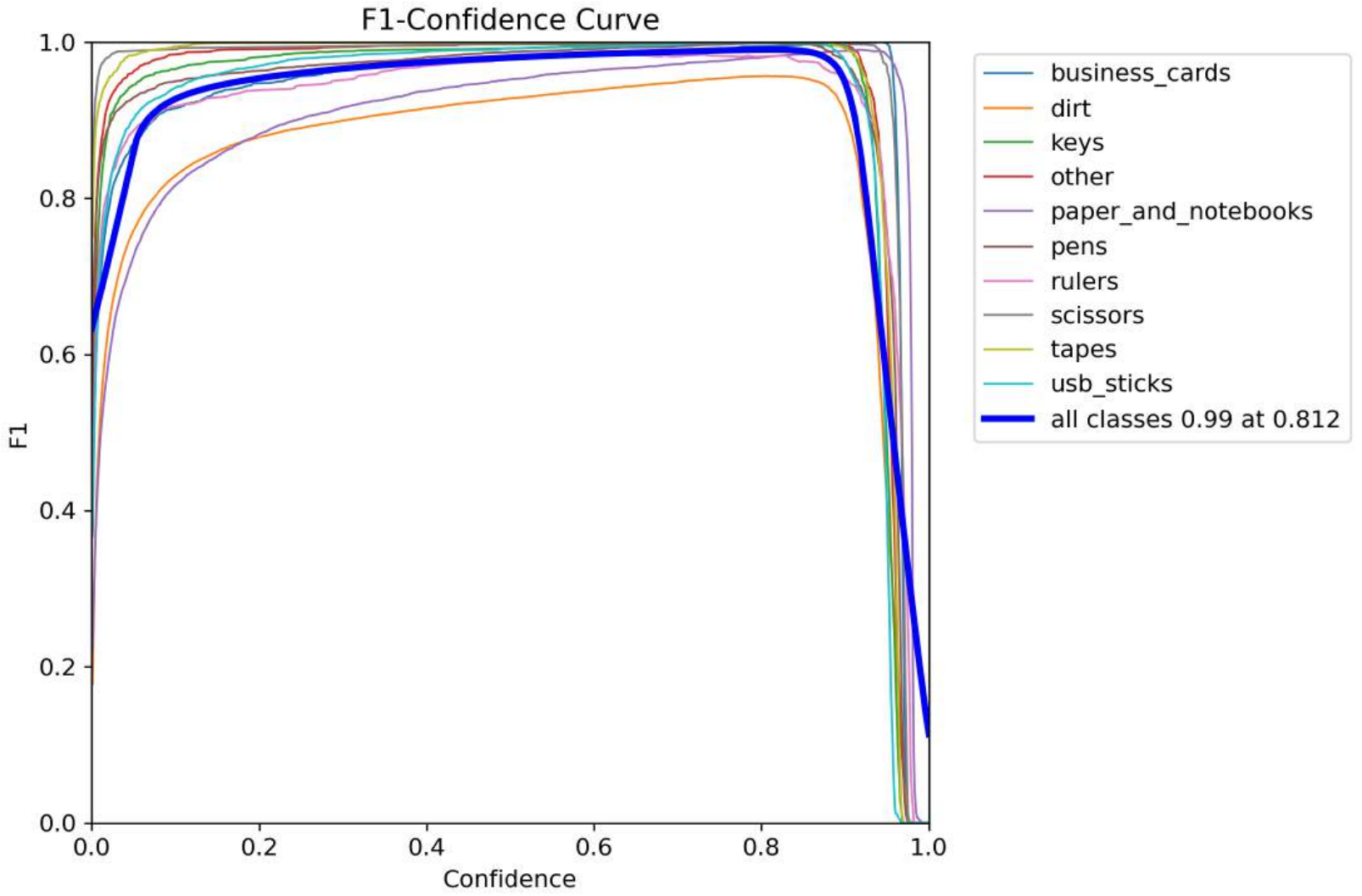
Yolov11x - GPU

- **NVIDIA-SMI & Driver**
  Version: 550.144.03
  CUDA: 12.4
- **GPU 0: NVIDIA GeForce RTX 3090 Ti**
  Memory: 24.0 GB
- **GPU 1: NVIDIA GeForce RTX 3090**
  Memory: 24.0 GB

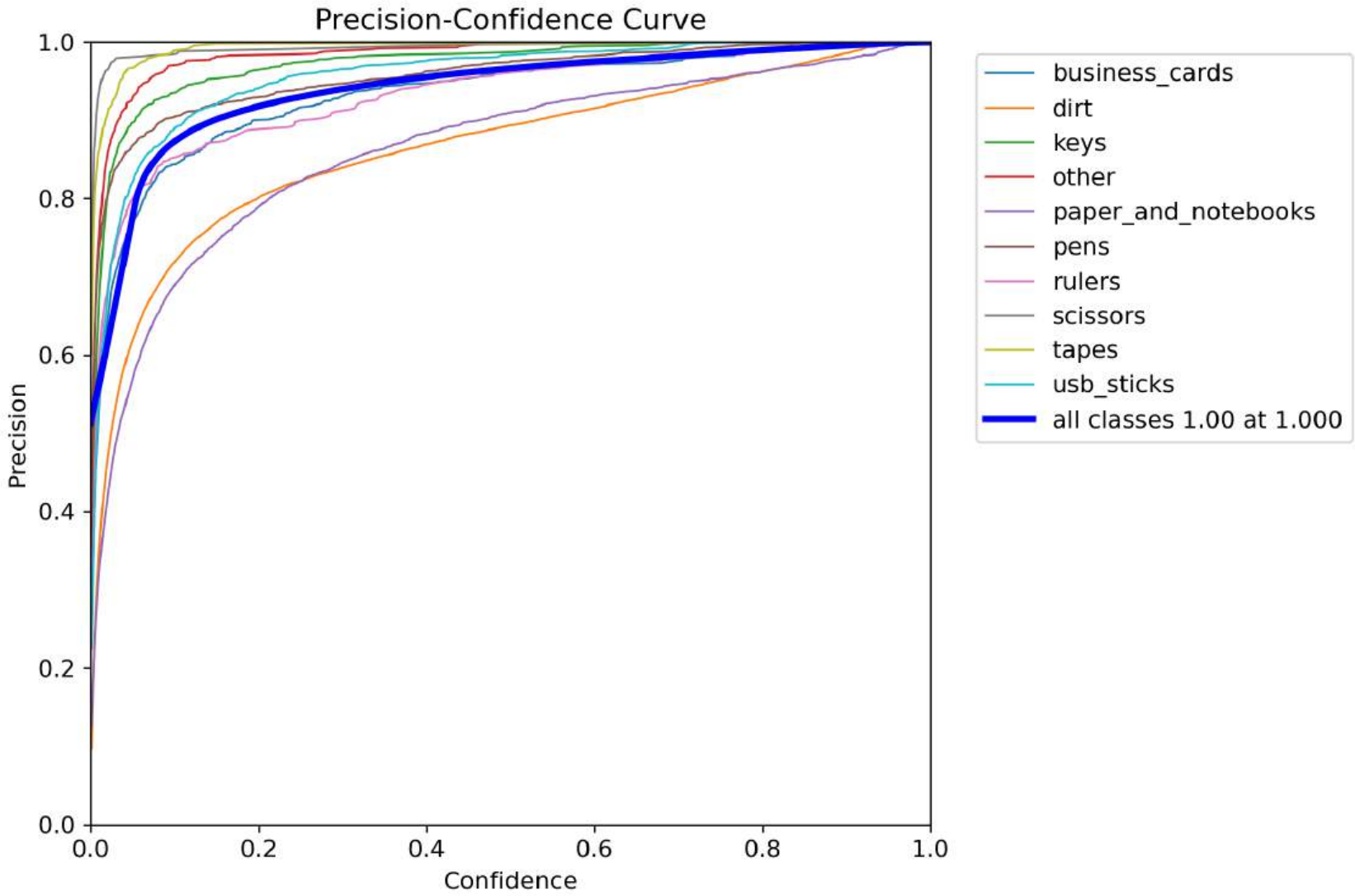
YOLOv11- Loss & mAP



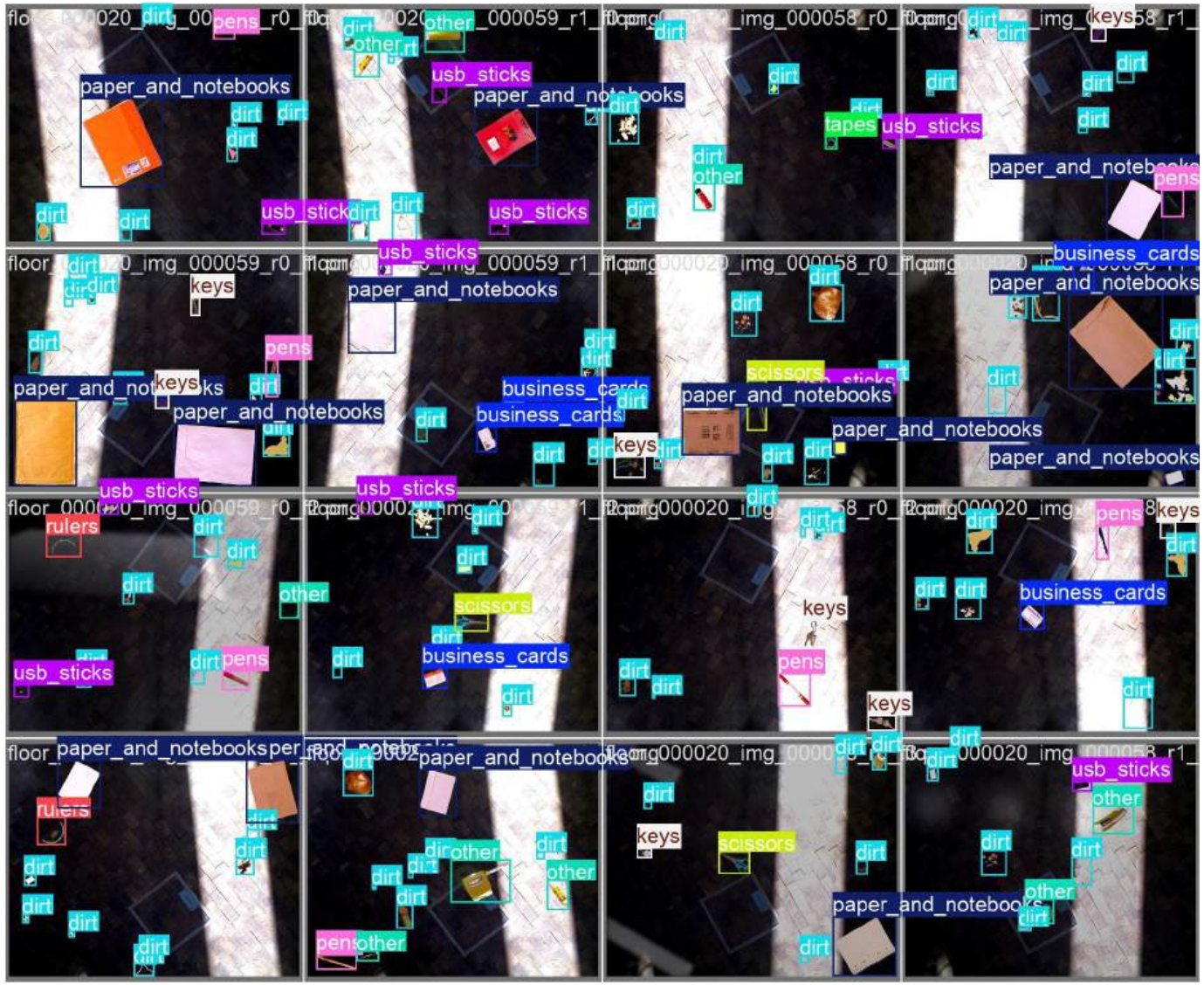
F1-Confidence Curve



Precision - Confidence Curve



Ground Truth



Prediction


paper_and_notebooks 1.0
usb_sticks 1.0
paper_and_notebooks 0.3
business_cards 1.0
keys 0.9
usb_sticks 0.4
paper_and_notebooks 0.9
usb_sticks 0.9
rulers 0.9
business_cards 0.3
keys 1.0
pens 1.0
business_cards 0.4
usb_sticks 0.3
business_cards 0.8
usb_sticks 0.9
paper_and_notebooks 0.4
paper_and_notebooks 0.8
paper_and_notebooks 0.9
paper_and_notebooks 0.3
pens 0.9

Optimization (Hardware & Throughput)

Optimization	What it does	Why it helps
Device= 0,1	Runs training on GPUs 0 and 1 in parallel, splitting each batch	Roughly halves epoch time compared to a single GPU
Workers= 8	8 CPU subprocesses	Keeps the GPUs fed, reduces idle time
Cache= Disk	**Caches the dataset uncompressed to disk** on the first epoch	**Subsequent epochs load images much faster**
Amp= true	Mixed Precision	**Cuts memory usage and provides 2–3× speedups** by leveraging GPU Tensor Cores
Imgsz=1024	Trains on **1024×1024-pixel inputs**	**Learns finer details (small objects)** at the cost of more compute per image.

Optimization (Training Stability & Convergence Optimizations)

Optimization	What it does	Why it helps
batch=16	Sets **16 images per gradient update**	Leads to **smoother gradient estimates**, resulting in **more stable training**
patience=10	Early-stopping: training **stops if validation loss doesn't improve for 10 epochs**	**Saves time and avoids over-fitting**
warmup_epochs=5	**Gradually ramps up the Learning Rate (LR) from near-zero to its initial value (lr0)** over the first 5 epochs	**Prevents unstable, large gradient steps** early on, especially with large batches.
lrf=0.1	**Decays the LR to lr0 × 0.1** (0.0001) by the end of training.	**Learns finer details (small objects)** at the cost of more compute per image.

Optimization (Data Augmentation)

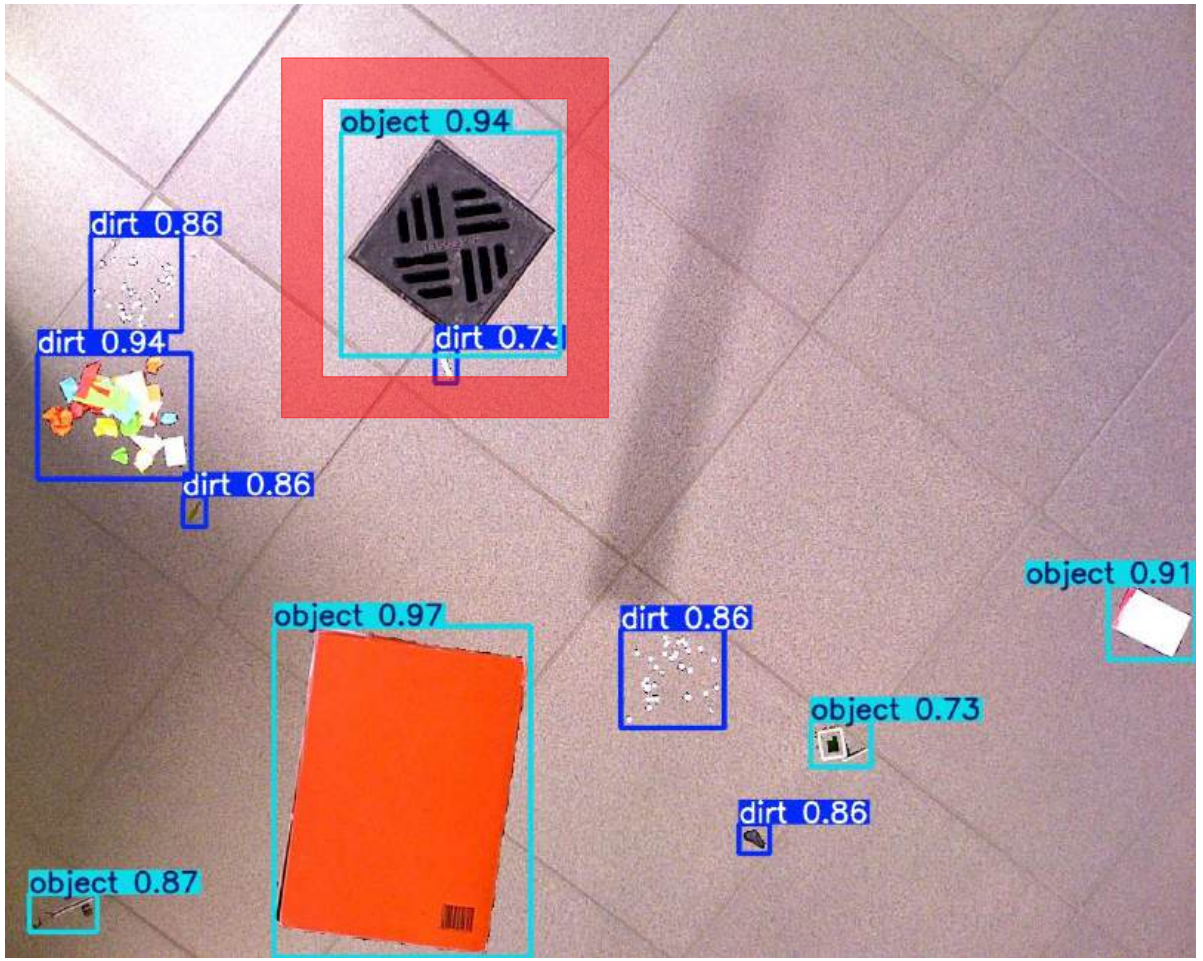
Optimization	What it does	Why it helps
mosaic=1.0	**Always stitches four random images into one composite**	**Exposes objects at varied scales and contexts**; **increases batch diversity**
copy_paste=0.5	With 50% probability, **pastes object crops from one image onto another**	**Boosts under-represented classes** and creates novel scenes
mixup=0.2	20% of the time, **blends two images and their labels**	**Smooths decision boundaries**, improving generalization to mixed inputs
auto_augment=randaugment	Applies a **random sequence of color, contrast, and geometric tweaks** discovered by RandAugment	**Diversifies the training set** with minimal manual tuning

Integration of Webcam

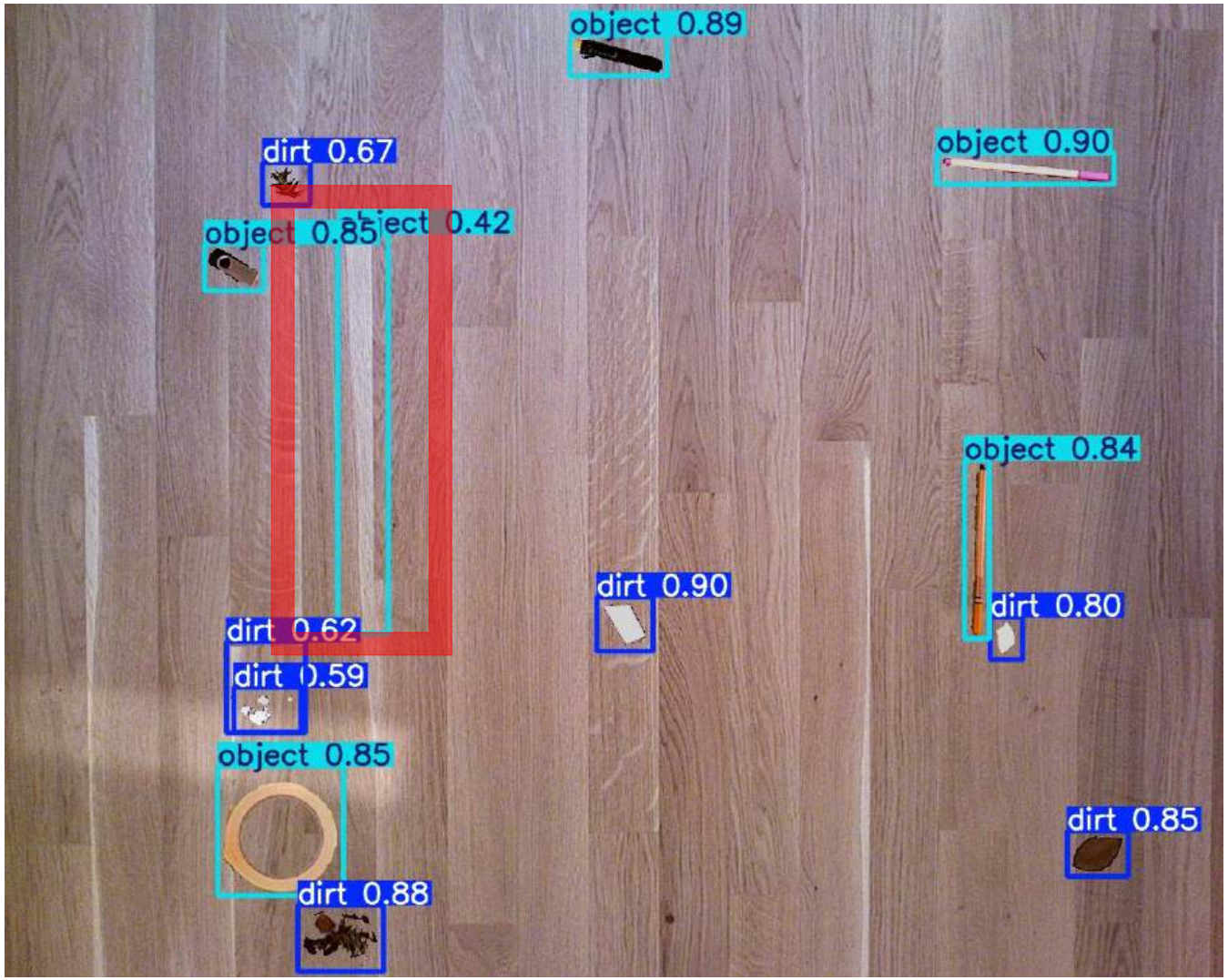
- **Isolated Python environment:**
  Created a dedicated virtual environment and installed all required libraries (Ultralytics, CoreMLTools, OpenCV) to ensure consistency and reproducibility.
- **Camera detection setup:**
  Programmatically scanned available video devices to identify the correct camera index—whether internal webcam or external/Continuity Camera.
- **Real-time detection pipeline:**
  Streamed frames from the selected camera into the object-detection model and overlaid live inference results for instant visual feedback.

Recording of Webcam

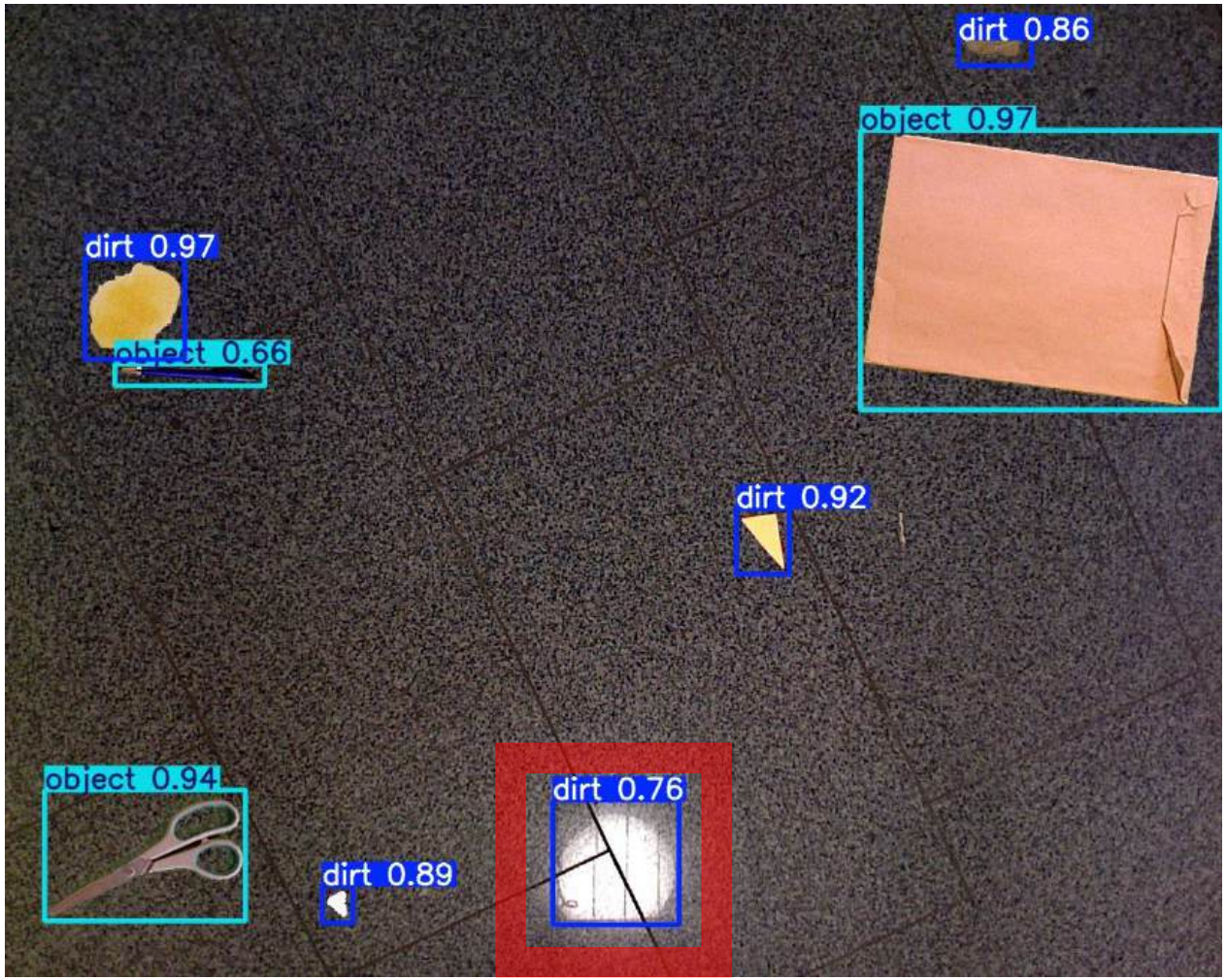
Limitations - 1



Limitations - 2



Limitations - 3



Suggestions for the future

- **Leverage faster architectures**
  - Use lightweight backbones (e.g. MobileNetV2) to speed up both training and on-board inference
  - Enables more frequent model updates and real-time dirt detection
- **Go beyond static-image training**
  - Live trials show dirt patches that mimic object shapes often get misclassified
  - Shape-based confusion cannot be resolved by image data alone
- **Incorporate salience measurement**
  - Add a saliency module to highlight truly “interesting” regions
  - Helps distinguish ambiguous dirt from office items and cuts down false positives

Conclusion

- **Slide Title: YOLOv11 Optimization & Results**
- **Top Model:** YOLOv11 achieved the highest performance (**mAP**)
- **Epoch Tuning:** Extended training boosted mAP—however causing overfitting
- **Hardware Acceleration:** GPU-based training dramatically reduced runtimes
- **Dual Implementation:** Successfully trained and implemented both YOLOv8 & YOLOv11
- **Data & Hyperparameter Scaling:** Larger dataset + targeted optimizations drove best results

Feedback after Presentation

- For Webcam, use a stabilization model

(if dirt is being detected 90% of the time in one spot, show that)

- NMS (Non-Maximum Suppression)

- Non-Maximum Suppression filters out overlapping boxes by keeping only the highest confidence prediction when two boxes overlap above the IoU threshold.

- The confidence threshold further weeds out low-score detections

(Source: Presentation by P05)